

A Differentially Private SQL Middleware for Repeated Aggregate Queries

CMP653 Project Milestone Report

Refik Can Oztas
Hacettepe University
Ankara, Turkiye
N25142279

Abstract

Aggregate SQL interfaces are often assumed to be safe because they expose only summary statistics rather than raw tuples. This assumption is misleading: repeated or overlapping aggregates can leak sensitive information through differencing and reconstruction attacks, and repeated query workloads quickly consume privacy budget under naive composition. This milestone report presents a lightweight differentially private SQL middleware for repeated aggregate workloads. The system supports a scoped but practical SQL subset consisting of COUNT, SUM, and AVG with simple WHERE clauses and optional GROUP BY. Its main contribution is a workload-aware budget ledger with exact-repeat caching, which allows a previously released noisy answer to be reused at zero additional privacy cost by the post-processing property of differential privacy. The report contributes a threat model, a focused review of related work, formal sensitivity and accounting analysis, a concrete middleware architecture on top of PostgreSQL, and a planned evaluation framework over TPC-H-style workloads. The current artifact is implementation-complete at the middleware layer, includes 28 passing unit tests, and is ready for the full benchmark campaign.

1 Introduction

Organizations routinely expose aggregate query interfaces to analysts, dashboards, and reporting tools. A common assumption is that counts, sums, and averages are inherently safe because they do not reveal raw tuples. This intuition is false. Privacy leakage emerges not only from one answer, but from how multiple answers combine across time.

A simple differencing attack illustrates the problem. Suppose an analyst can issue the following two queries on a hospital table:

```
SELECT COUNT(*)
FROM patients
WHERE zipcode = '06897' AND disease = 'HIV';

SELECT COUNT(*)
FROM patients
WHERE zipcode = '06897' AND disease = 'HIV'
AND age <> 43;
```

If the first query returns 12 and the second returns 11, the analyst can infer that the unique 43-year-old patient in that zipcode has HIV. Each query reveals only an aggregate, yet their difference leaks an individual fact. More generally, Dinur and Nissim showed that too many sufficiently accurate aggregate answers enable large-scale database reconstruction [2]. In practical systems, the modern version of this threat appears as repeated analytical workloads:

analysts ask many related queries, and privacy loss accumulates across the sequence.

Differential privacy provides the standard formal defense. However, deploying it inside a SQL system is not merely a mathematical exercise. A practical system must determine sensitivity, add calibrated noise, track cumulative privacy loss, and manage repeated workloads without wasting budget. This project studies that systems problem in a deliberately scoped setting. Instead of building a full private DBMS, it implements a middleware layer between the client and PostgreSQL for repeated aggregate queries.

The central research question is:

Can workload-aware privacy accounting with exact-repeat reuse provide better utility–budget trade-offs than naive per-query composition for repeated aggregate workloads?

The contribution is architectural rather than theorem-driven. The paper combines supported-query validation, sensitivity-aware noise injection, stateful budget accounting, and exact-repeat reuse in a single experimental framework appropriate for a database systems course project.

2 Background and Related Work

Differential privacy was introduced by Dwork et al. [3] and later systematized by Dwork and Roth [4]. Their framework formalizes privacy through neighboring databases, sensitivity, and randomized mechanisms such as the Laplace mechanism. For repeated query workloads, composition theorems are central because privacy loss accumulates across releases.

The attack model motivating this work predates differential privacy. Denning studied tracker attacks in statistical databases [1], showing how overlapping aggregates can isolate individual information. Dinur and Nissim established a stronger impossibility result: too many accurate subset-sum style answers can reveal a large portion of the database [2]. Modern surveys continue to emphasize that aggregate-release systems remain vulnerable without formal protection [5].

Several systems have brought differential privacy closer to relational query processing. PINQ integrated privacy tracking into a LINQ-like interface [6]. Johnson et al. [7] studied practical differential privacy for SQL queries, while PrivateSQL [8] and DOP-SQL [12] moved toward broader SQL-supporting systems. These works show that private query answering is an architectural and systems problem, not just a mechanism-design problem.

Recent work also highlights that relational structure matters. Residual Sensitivity studied multi-way joins under differential privacy [9], while R2T exploited foreign-key structure for tighter bounds [10]. These techniques are powerful but scope-expanding. The present project therefore targets a narrower and cleaner setting: repeated single-table aggregate queries with explicit workload-level accounting.

The most conceptually related work is *Better than Composition* [11], which showed that multiple relational queries can sometimes be answered more accurately than naive independent composition suggests. Our project shares the same motivation but takes a simpler systems path. Instead of relational joint optimization, it studies exact-repeat reuse and ledger-based accounting as a practical improvement for repeated workloads commonly seen in dashboards and exploratory analytics.

3 Threat Model and Methodology

We assume a relational dataset D stored in PostgreSQL and a user who submits supported aggregate queries through the middleware. The adversary is any analyst who can adaptively issue supported queries and observe the sequence of outputs. The goal is to limit what the analyst can infer about any one tuple while preserving useful answers.

DEFINITION 1 (NEIGHBORING DATABASES). Two databases D and D' are neighbors, written $D \sim D'$, if they differ in exactly one tuple. This report adopts the unbounded tuple-level model.

DEFINITION 2 (ϵ -DIFFERENTIAL PRIVACY). A randomized mechanism $\mathcal{M}$ satisfies ϵ -differential privacy if for every pair of neighboring databases $D \sim D'$ and every measurable set of outputs S ,

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S].$$

The middleware supports a restricted SQL subset: COUNT, SUM, and AVG over a single table, with optional GROUP BY and simple conjunctive predicates. This restriction keeps sensitivity analysis transparent and makes the privacy policy auditable.

For the supported query class, global L_1 sensitivity is

$$\Delta f = \max_{D \sim D'} |f(D) - f(D')|.$$

The middleware uses the following assumptions:

- **COUNT:** $\Delta f = 1$.
- **SUM:** $\Delta f = B$ when the aggregated attribute is clipped to $[0, B]$.
- **AVG:** implemented through bounded noisy SUM and noisy COUNT.

The private release uses the Laplace mechanism:

$$\tilde{f}(D) = f(D) + \eta, \quad \eta \sim \text{Lap}(\Delta f / \epsilon_q).$$

Thus, smaller ϵ_q yields stronger privacy but larger expected error.

The central systems idea is workload-aware budget accounting. Let the total workload budget be ϵ_{total} and the nominal per-cache-miss cost be ϵ_q . Under naive accounting, a workload of k queries spends

$$\epsilon_{\text{naive}}^{(k)} = k\epsilon_q.$$

Under workload-aware accounting with exact-repeat caching, only unique cache-miss queries consume budget:

$$\epsilon_{\text{wa}}^{(k)} = u_k \epsilon_q,$$

Algorithm 1 DP middleware for repeated aggregate queries

Require: query q , remaining budget ϵ_{rem} , per-query budget ϵ_q

- 1: Parse and validate q against the supported SQL subset
- 2: Normalize q to obtain a canonical key
- 3: **if** key exists in cache **then**
- 4: Return cached noisy answer
- 5: **end if**
- 6: **if** $\epsilon_{\text{rem}} < \epsilon_q$ **then**
- 7: Reject query
- 8: **end if**
- 9: Analyze the aggregate type and determine sensitivity Δf
- 10: Execute the exact query on PostgreSQL and obtain result r
- 11: Sample noise $\eta \sim \text{Lap}(\Delta f / \epsilon_q)$
- 12: Compute $\tilde{r} \leftarrow r + \eta$
- 13: Store $\tilde{r}$ in cache and append ledger entry
- 14: Decrease remaining budget by ϵ_q
- 15: Return $\tilde{r}$

where u_k is the number of unique exact queries among the first k requests.

PROPOSITION 1 (PRIVACY OF EXACT-REPEAT REUSE). Assume a query q is answered once using an ϵ_q -differentially private mechanism. Returning the same noisy output for subsequent identical requests of q consumes no additional privacy budget.

Proof sketch. The repeated return is a function only of an already released differentially private output. By the post-processing property, no additional privacy loss is incurred.

This immediately yields the budget benefit. For k repeated requests of the same exact query, naive accounting spends $k\epsilon_q$, whereas workload-aware reuse spends only ϵ_q . The budget savings ratio is therefore $1 - 1/k$.

Algorithm 1 summarizes the middleware logic.

4 Implementation, Data, and Experimental Design

The prototype is implemented in Python as a middleware layer on top of PostgreSQL. Its main components are a SQL parser and validator, a query analyzer, a Laplace mechanism layer, a stateful budget ledger, a template and exact-query matcher, an orchestrator, and a command-line interface. The current artifact also includes 28 passing unit tests, TPC-H schema DDL, a Python data generator, four workload families, and a benchmark runner. This means the main remaining effort is experimental execution rather than core architecture development.

4.1 Dataset: TPC-H as a Realistic Analytical Substrate

TPC-H is used as the source schema because it models a generic wholesale supplier whose data (orders, customers, line items, suppliers, parts) resembles the kind of sensitive commercial data a real analytics platform would host. Two tables dominate the workloads used here:

- **lineitem**: the largest fact table, with one row per ordered line item. Relevant attributes include `l_extendedprice` (revenue), `l_discount`, `l_quantity`, `l_shipdate`, `l_returnflag`, and `l_shipmode`.
- **orders**: one row per customer order, with `o_totalprice`, `o_orderdate`, `o_orderpriority`, and `o_orderstatus`.

Individual rows in these tables can be sensitive: a specific line item can identify a particular customer’s purchase behaviour, and an order row reveals total spend for one account. An adversary who can freely issue COUNT and SUM queries with arbitrary WHERE clauses can, in principle, isolate a single customer’s activity through differencing attacks analogous to the HIV example in Section 1. TPC-H therefore serves as a realistic stand-in for a commercial analytical dataset while remaining easy to deploy and reproduce in PostgreSQL. The project does not attempt to reproduce official TPC-H benchmark queries; TPC-H is used only as a relational substrate over which repeated aggregate templates are generated.

4.2 Query Templates and Use Cases

Five templates are used throughout the evaluation. Each is grounded in a concrete analyst use case so that its role in the benchmark — and the privacy risk it illustrates — is explicit.

T1: Shipped-items count (revenue dashboard tile). A logistics dashboard shows “number of line items shipped since a given date.” The tile refreshes on every page load, so the same query text is issued repeatedly by many viewers.

```
SELECT COUNT(*)
FROM lineitem
WHERE l_shipdate >= DATE '1995-01-01';
```

Sensitivity is $\Delta f = 1$. Because the query text never changes, a naive DP system spends ϵ_q on every refresh. Under exact-repeat reuse, the dashboard costs ϵ_q *once* and then serves the cached noisy count for all subsequent refreshes. This is the clearest example of where workload-aware accounting pays off.

T2: Orders by priority (parametric filter). An operations analyst investigates backlog by priority level and issues the same template with different literals: ‘1-URGENT’, ‘2-HIGH’, ‘3-MEDIUM’, etc.

```
SELECT COUNT(*)
FROM orders
WHERE o_orderpriority = '1-URGENT';
```

Sensitivity is $\Delta f = 1$. Each distinct literal forms a separate cache key, so exact-repeat reuse helps only when an analyst revisits the same priority value (which is common — “urgent” is the most-clicked tile). This template measures how much of a realistic parametric workload resolves to repeated exact queries rather than genuinely new ones.

T3: Revenue by return flag (grouped sum). A finance team monitors revenue split across return statuses (R = returned, A = accepted, N = none), typically once per day across the same shipping window.

```
SELECT l_returnflag, SUM(l_extendedprice)
FROM lineitem
WHERE l_shipdate >= DATE '1995-01-01'
GROUP BY l_returnflag;
```

Sensitivity is determined by the clipping bound B on `l_extendedprice`; with $B = 10^5$, each group’s noisy sum has sensitivity B . A GROUP BY with g groups produces g simultaneous releases, so the effective per-query cost is $g\epsilon_q$ unless groups are privacy-budgeted jointly. This template stresses the accounting layer: it is both the most expensive single query in the workload and a strong candidate for reuse, because the dashboard re-issues it on every refresh.

T4: Average order value by status (AVG via SUM/COUNT). A CFO dashboard tracks average order value broken down by order status.

```
SELECT o_orderstatus, AVG(o_totalprice)
FROM orders
WHERE o_orderdate >= DATE '1996-01-01'
GROUP BY o_orderstatus;
```

The middleware implements AVG as a noisy SUM divided by a noisy COUNT, so the query consumes $2\epsilon_q$ per group. This template is included specifically to show that AVG is *not* free even though it “looks like” a single aggregate — a point that a budget ledger makes visible but a stateless DP library hides.

T5: Progressive drill-down (narrowing predicates). An analyst investigating anomalous revenue narrows a query step by step, each step a stricter predicate:

```
-- Step 1
SELECT SUM(l_extendedprice)
FROM lineitem
WHERE l_shipdate >= DATE '1995-01-01';

-- Step 2
SELECT SUM(l_extendedprice)
FROM lineitem
WHERE l_shipdate >= DATE '1995-01-01'
  AND l_returnflag = 'R';

-- Step 3
SELECT SUM(l_extendedprice)
FROM lineitem
WHERE l_shipdate >= DATE '1995-01-01'
  AND l_returnflag = 'R'
  AND l_shipmode = 'AIR';
```

These three queries are closely related — each result is a subset of the previous — but they are not exact repeats, so caching does not help. This template exists precisely to measure the *limits* of exact-repeat reuse: a drill-down session should consume full budget and the evaluation should show it. It also represents the differencing-attack pattern most clearly: successive narrowing is essentially how a tracker attack is constructed.

4.3 Workload Families

The five templates are combined into four workload families, each targeting a different axis of the research question.

- **W1: Repetitive (dashboard refresh).** T1 issued 100 times in a row. Simulates a tile that auto-refreshes. Expected behaviour: naive spends $100\epsilon_q$; workload-aware spends ϵ_q .
- **W2: Parametric (filter sweep).** T2 issued with literals drawn from a Zipf-like distribution over priority values. Expected behaviour: reuse helps proportionally to the skew of the distribution.
- **W3: Diverse (mixed analytics).** Interleaved mix of T1–T4. Simulates multiple dashboards sharing one budget. Expected

Table 1: Workload families and the templates they exercise.

Workload	Templates	Purpose
W1	T1	Measures budget savings from exact-repeat reuse
W2	T2	Tests template stability under changing literals
W3	T1–T4	Approximates exploratory analytics with mixed aggregates
W4	T5	Simulates analyst drill-down; expected worst case for caching

behaviour: savings come almost entirely from T1 and T3 (the repeating tiles), not from T2 and T4.

- **W4: Progressive (drill-down).** T5 sequence, repeated with different starting dates. Expected behaviour: exact-repeat reuse provides little benefit; the workload exposes the residual budget problem that motivates future work.

Table 1 summarizes this mapping.

The planned benchmark compares three modes: exact PostgreSQL execution, naive DP execution, and workload-aware DP execution. Utility will be measured using absolute error, relative error, MAE, RMSE, and answer quality rate. Budget efficiency will be evaluated using queries-per-epsilon, budget exhaustion point, cache hit rate, and budget savings ratio. Performance will be measured using median latency, p95 latency, throughput, and middleware overhead. A later privacy-oriented analysis will include membership-inference-style and reconstruction-style experiments.

5 Current Findings and Discussion

The most important result at the current stage is architectural and analytical rather than a fully populated benchmark table. The middleware is implementation-complete for the target SQL subset, and the workload-aware path already behaves differently from naive composition in a way that directly matters to the research question.

For an exact repeated query, the first request is parsed, validated, executed on PostgreSQL, privatized with Laplace noise, and stored in the cache. A subsequent identical request returns the same noisy output without spending additional privacy budget. This is not a heuristic optimization; it follows directly from the post-processing property of differential privacy. Therefore, the workload-aware mechanism is strictly more budget-efficient than a naive stateless system on any workload with exact repetition.

Table 2 shows a toy trace with $\epsilon_q = 0.2$.

The example is small, but it captures the central hypothesis of the paper: repeated workloads create an opportunity for systems-level savings that naive composition ignores. The current artifact already validates the mechanism needed to test that hypothesis at scale.

At the same time, several limitations remain. First, the current scope excludes joins, nested queries, and arbitrary SQL expressions. Second, exact-repeat reuse assumes a stable database snapshot or a clear cache invalidation policy. Third, the strongest empirical claims still depend on the full benchmark campaign. For that reason, this

Table 2: Illustrative privacy-budget trace for a repeated workload.

Step	Query ID	Naive spent	Workload-aware spent
1	Q_A	0.2	0.2
2	Q_B	0.4	0.4
3	Q_B	0.6	0.4
4	Q_C	0.8	0.6
5	Q_B	1.0	0.6

Table 3: Main metrics used in the planned evaluation.

Category	Representative metrics
Utility	MAE, RMSE, MRE, AQR
Budget efficiency	Queries-per-epsilon, budget exhaustion point, cache hit rate, budget savings ratio
Performance	Median latency, p95 latency, throughput, middleware overhead
Privacy leakage	Membership inference accuracy, empirical privacy loss, reconstruction error

report makes no fabricated performance claims. Instead, it provides analytical and artifact-level evidence, together with a benchmark design ready for execution.

6 Evaluation Metrics, Validity, and Remaining Work

The planned evaluation is organized around four categories so that the paper is judged as a database systems study rather than only a privacy note. **Utility** will be measured using mean absolute error (MAE), root mean squared error (RMSE), mean relative error (MRE), and an answer quality rate (AQR) defined as the fraction of queries whose relative error remains below a chosen tolerance. **Budget efficiency** will be measured using queries-per-epsilon, budget exhaustion point, cache hit rate, and budget savings ratio. **Performance** will be measured using median latency, p95 latency, and middleware overhead relative to exact PostgreSQL execution. Finally, **privacy leakage** will be examined through membership-inference-style probes and simplified reconstruction-style simulations on repeated releases.

These metrics matter because utility alone does not tell the full story. A mechanism with low error but unsustainably high privacy cost is not useful in practice, just as a mechanism with low budget consumption but unusable answers is not a meaningful result. The project therefore evaluates the utility–privacy–performance trade-off jointly.

Several validity concerns must be acknowledged. **Internal validity** depends on the correctness of the parser, sensitivity rules, and budget ledger. The current prototype reduces this risk through unit tests, but the strongest validation will come from full end-to-end experiments and manual inspection of noisy outputs on representative workloads. **External validity** is limited by the scoped SQL

Table 4: Artifact status at the time of this milestone report.

Component	Status
SQL parser and validator	Implemented
Sensitivity analyzer	Implemented for COUNT, SUM, AVG
Budget ledger and cache	Implemented
Workload families W1–W4	Defined
Benchmark runner	Implemented
Large-scale experiment execution	Pending final run
Leakage experiments	Planned

subset. Results on single-table aggregates may not generalize directly to joins, nested queries, or production data pipelines with updates and concurrency. **Construct validity** depends on whether the chosen workloads capture realistic analytical behavior. The four workload families are intended to approximate common dashboard and drill-down patterns, but they remain a simplified proxy rather than logs from a production analytics system.

The remaining work is now concentrated in experimentation rather than implementation. First, PostgreSQL must be populated at the planned TPC-H scale factors and the full benchmark grid must be executed across workload families, privacy settings, and repeated trials. Second, the resulting measurements must be converted into publication-quality figures and comparison tables. Third, the privacy-leakage analysis must be completed so that the final paper evaluates not only answer distortion but also whether the privacy mechanism meaningfully weakens inference attacks.

7 Conclusion

This project studies privacy-preserving SQL processing as a database systems problem rather than only a mechanism-design problem. Aggregate queries are not inherently safe: repeated or overlapping answers can leak sensitive information, and repeated analytical workloads consume privacy budget quickly under naive composition. In response, the project builds a lightweight differentially private SQL middleware for repeated aggregate queries.

The core idea is simple but meaningful: exact-repeat reuse should be treated as a first-class systems optimization because it preserves utility while consuming no additional privacy budget. The current artifact already implements the required architecture, sensitivity rules, and accounting logic on top of PostgreSQL, and it is ready for large-scale evaluation over TPC-H-style workloads. This makes the project a good fit for the course objectives: it combines formal analysis, practical implementation, and comparative evaluation within a realistic relational setting.

References

- [1] Dorothy E. Denning. 1979. The Tracker: A Threat to Statistical Database Security. *ACM Transactions on Database Systems* 4, 1 (1979), 76–96.
- [2] Irit Dinur and Kobbi Nissim. 2003. Revealing Information while Preserving Privacy. In *Proceedings of the 22nd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*. 202–210.
- [3] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Proceedings of the Third Theory of Cryptography Conference (TCC)*. 265–284.
- [4] Cynthia Dwork and Aaron Roth. 2014. *The Algorithmic Foundations of Differential Privacy*. Foundations and Trends in Theoretical Computer Science 9, 3–4 (2014), 211–407.
- [5] Cynthia Dwork, Adam Smith, Thomas Steinke, Jonathan Ullman, and Salil Vadhan. 2017. Exposed! A Survey of Attacks on Private Data. *Annual Review of Statistics and Its Application* 4 (2017), 61–84.
- [6] Frank McSherry. 2009. Privacy Integrated Queries: An Extensible Platform for Privacy-Preserving Data Analysis. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*. 19–30.
- [7] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards Practical Differential Privacy for SQL Queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539.
- [8] Ios Kotsogiannis, Saeed Meiser, Jose M. del Alamo, Aastha Chaudhuri, Mohammad Hayyeri, Jerome Miklau, Michael Hay, and Ashwin Machanavajjhala. 2019. PrivateSQL: A Differentially Private SQL Query Engine. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1371–1384.
- [9] Wei Dong and Ke Yi. 2021. Residual Sensitivity for Differentially Private Multi-Way Joins. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 432–445.
- [10] Wei Dong, Juanru Fang, Ke Yi, Yuchao Tao, and Ashwin Machanavajjhala. 2022. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. 759–772.
- [11] Wei Dong, Dajun Sun, and Ke Yi. 2023. Better than Composition: How to Answer Multiple Relational Queries under Differential Privacy. *Proceedings of the ACM on Management of Data* 1, 2 (2023), Article 123.
- [12] Jianzhe Yu, Wei Dong, Juanru Fang, Dajun Sun, and Ke Yi. 2024. DOP-SQL: A General-purpose, High-utility, and Extensible Private SQL System. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4385–4388.